AI Personal Library

Production System Documentation

A highly scalable, containerized document ingestion and Semantic Search system. Engineered for local CPU deployment under resource-constrained desktop environments.

Architecture Format:	Multi-Container Microservices (Docker Compose)
Database Systems:	SQLite (WAL Mode Enabled) & ChromaDB Vector Store
Retrieval Framework:	Hybrid Search (BM25 + Dense Cosine) with Reranker
Large Language Model:	gemini-3.1-flash-lite (via Google ADK)
Release Version:	v2.2 (Refactored)

AI Personal Library - Production System Documentation

Local CPU-Optimized RAG & Multi-Session Agentic Chat Application

1. Executive Summary & Overview

The AI Personal Library is a microservices-based, containerized Retrieval-Augmented Generation (RAG) platform. The application is designed to run entirely on resource-constrained consumer hardware (specifically optimized for a Windows 8GB RAM host CPU profile).

The platform monitors a local directory for incoming document uploads, automatically ingests and parses them page-by-page, detects document headings and layouts, constructs semantic chunks, and builds a dual-index search repository (combining keyword and concept vector indices).

An agentic reasoning loop (built on the Google ADK Framework using `gemini-3.1-flash-lite`) provides context-aware, cited answers through a responsive, multi-session React dashboard.

2. System Architecture

The platform follows a clean, modular microservices topology orchestrated using Docker Compose:

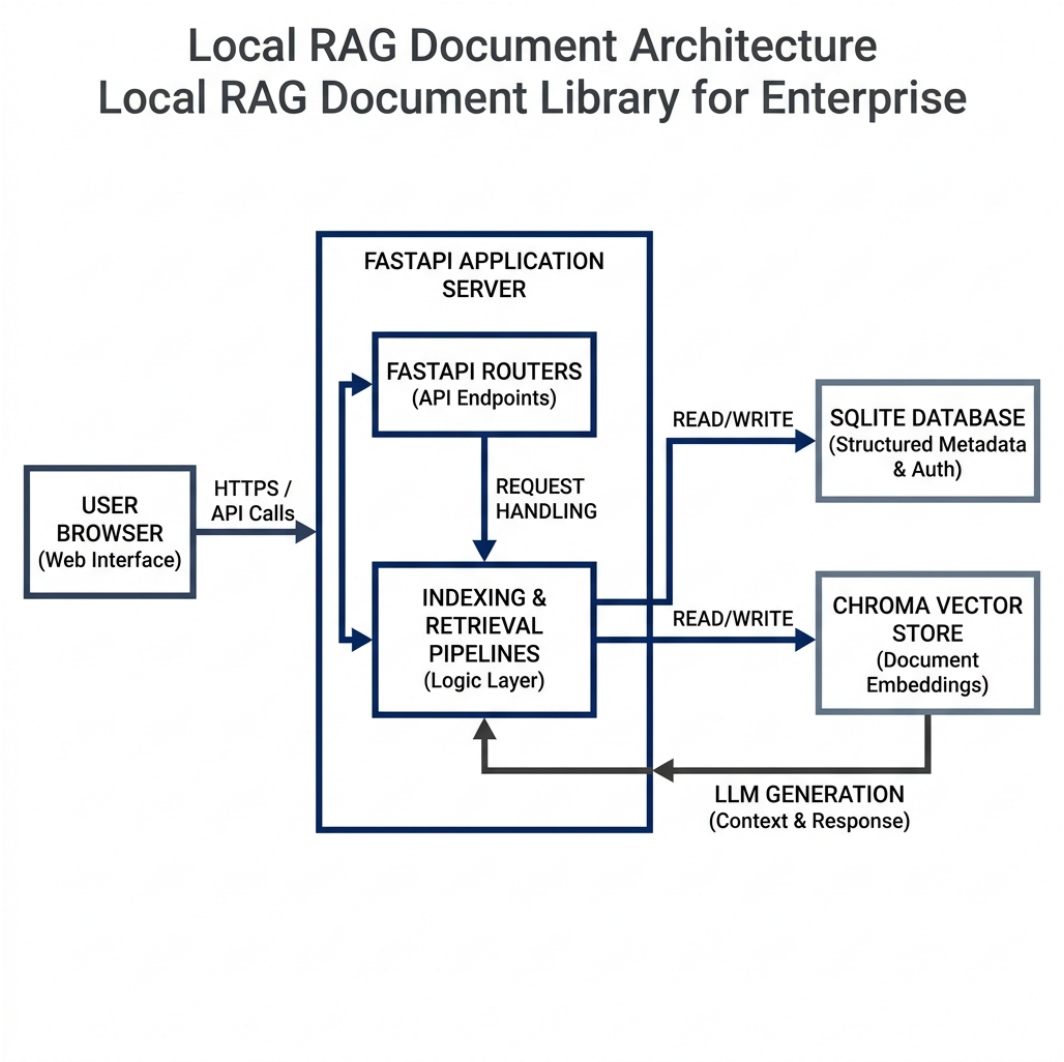

Figure: System Architecture Diagram

Core Architecture Components:

1. Frontend App (library_frontend): Runs inside a Node.js container, serving the React SPA dashboard via Vite. Configures a reverse-proxy mapping for `/api` queries to avoid CORS issues.

2. Backend Server (library_backend): Runs inside a Python container, orchestrating FastAPI routes and coordinating async worker pipelines.

3. Directory Watchdog Daemon: Monitors the host-mounted `./books` folder. Copying or dropping files directly on Windows triggers automatic processing without user commands.

4. Data Persistence Volumes: Maps Windows physical host folders to containers to persist indices and model weights across system rebuilds.

3. Data Ingestion & Pipeline Specification

The ingestion process is automated and executed sequentially in the background to prevent memory crashes:

```
[File Saved/Copied] ---> [Stability Checker] ---> [Acquire Lock] ---> [Extract Text]
                                                                          |
[Chroma Vector Store] <--- [Nomic Embeddings] <--- [Token Chunker] <--- [Section Split]
```

3.1 Directory Watcher & File Stability

The `watchdog` library listens for file creation signals. When a file is detected, a worker thread enters a verification loop to check if the file size remains identical for two consecutive checks. This prevents the parser from reading files that are still being copied.

3.2 Thread-Lock Protection

A global `indexing_lock = threading.Lock()` coordinates the indexing thread. If multiple files are uploaded, they are queued and processed sequentially. This prevents PyTorch from allocating concurrent matrix pools, safeguarding the 8GB host memory profile.

3.3 Text Extraction & Structural Annotation

* PDFs: Parsed page-by-page using `PyMuPDF` (`fitz`).
* EPUBs: Parsed using `EbookLib` and `BeautifulSoup4`.
* DOCX: Parsed using `python-docx`.
* Structure Detection: Font metrics, capitalization, and layout structures are evaluated to detect header coordinates. The document is annotated with structural boundaries so each text chunk knows which book chapter it belongs to.

3.4 Token Chunking & Context Prefixing

To prevent semantic dilution, text is chunked into overlapping segments of ~500 tokens using a sliding window. Each chunk is prefixed with structural parameters:

```
Document: [Title] | Section: [Chapter] | Page: [Page Number]
[Paragraph content...]
```

3.5 Vector & Keyword Index Storage

1. Vector Embeddings (ChromaDB): Text chunks are passed in batches of 16 to the local `nomic-embed-text-v1.5` model. It outputs a 768-dimensional float vector which is saved in ChromaDB's HNSW graph files.
2. Keyword Indexes (SQLite FTS5): Chunk text is stored in SQLite's virtual `chunk_fts` index table, enabling instant keyword search.

4. Query Retrieval Loop & Hybrid Search

The search process utilizes a Hybrid Search Pipeline to retrieve high-accuracy context for the agent:

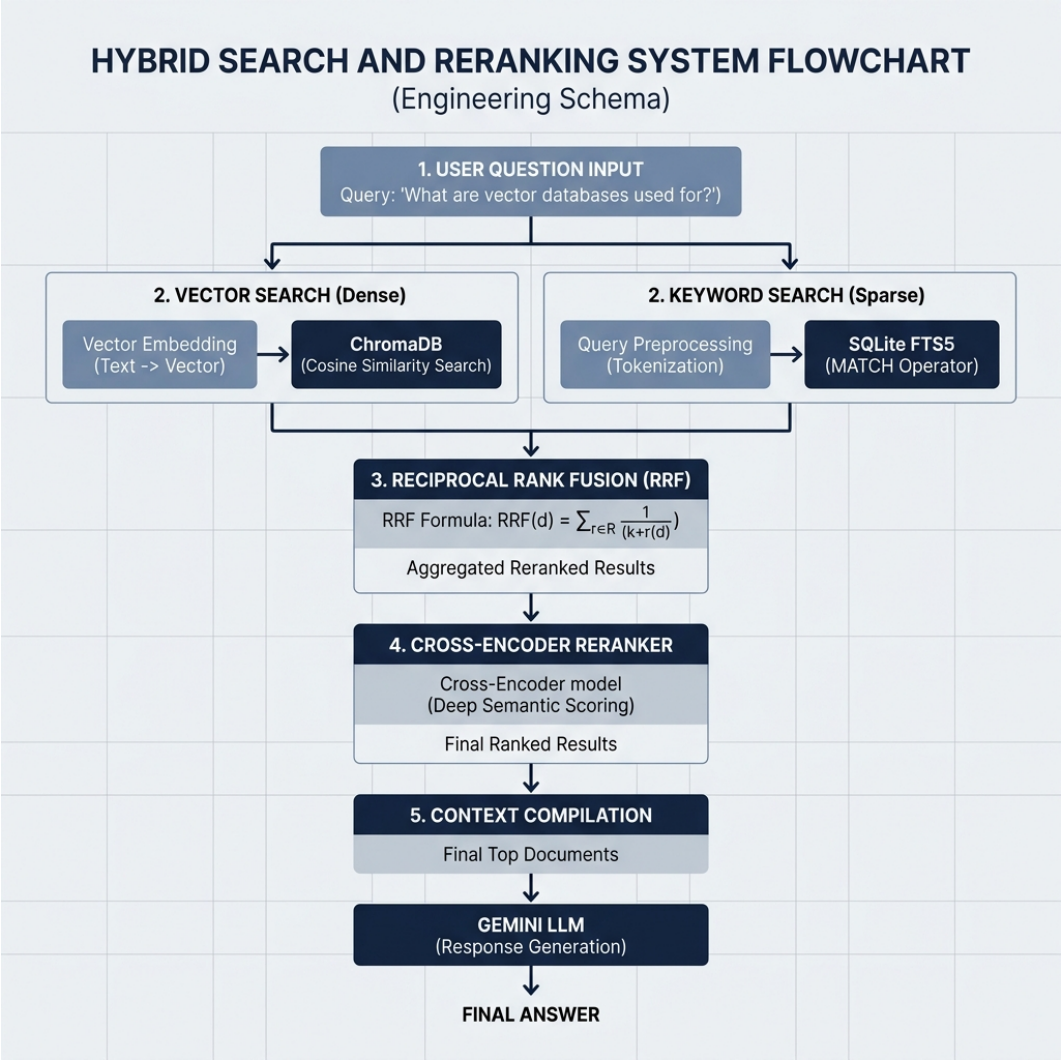

Figure: Hybrid Search & Reranking Flow

4.1 Hybrid Retrieval Components

* Dense Retrieval (ChromaDB): Converts query to a concept vector and finds matching nodes in the HNSW space based on cosine distance.

* Sparse Retrieval (SQLite FTS5): Executes SQL keyword query matching:

```
SELECT chunk_id, text FROM chunk_fts WHERE text MATCH :query LIMIT 8;
```

* Reciprocal Rank Fusion (RRF): Merges the results of both searches mathematically. It ranks the combined candidate list using the formula:

$$\text{Score} = \sum_{m \in M} \frac{1}{60 + \text{Rank}_m}$$

4.2 Cross-Encoder Reranking

The merged list is processed by the local Cross-Encoder model `ms-marco-MiniLM-L-6-v2`. It outputs a relevance score for each candidate chunk, sorting the most helpful passages to the top. The number of candidate chunks evaluated is capped by `RERANKER_CANDIDATE_LIMIT=8` in `.env` to prevent CPU resource limits.

4.3 Agent Reasoning (Gemini Integration)

The Google ADK Agent coordinates tool calls and structures the final response. It reads the top reranked chunks, verifies details, formats the markdown citations (e.g. `[Book Title, Page 4]`), and streams the output to the frontend.

5. API Endpoint Specifications

5.1 Status Check

* Endpoint: `GET /api/status`

* Response:


```
{
  "status": "healthy",
  "embedding_model": "nomic-ai/nomic-embed-text-v1.5",
  "reranker_model": "cross-encoder/ms-marco-MiniLM-L-6-v2",
  "statistics": {
    "total_books": 5,
    "completed": 5,
    "processing": 0,
    "queued": 0,
    "failed": 0
  }
}
```


5.2 List Books

* Endpoint: `GET /api/books`

* Response:


```
[
  {
    "id": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
    "title": "The Time Machine",
    "author": null,
    "format": ".pdf",
    "pages": 84,
    "file_size": 240432,
    "num_chunks": 112,
    "indexed_time": "2026-07-11T12:00:00.000000",
    "status": "completed",
    "error_message": null
  }
]
```


5.3 Upload Book

* Endpoint: `POST /api/upload`

* Payload: `multipart/form-data` with `file`

* Response:


```
{
  "status": "queued",
  "book_id": "e3b0c44298fc1c149afbf4...",
```

```
    "title": "The Time Machine"
  }
```

5.4 Delete Book

* Endpoint: `DELETE /api/books/{book_id}`

* Response:

```
{
  "status": "success",
  "message": "Deleted book 'The Time Machine'"
}
```

5.5 Chat Query

* Endpoint: `POST /api/chat`

* Payload:

```
{
  "query": "Who wrote the Time Machine?",
  "scope": "library",
  "book_ids": [],
  "session_id": "default"
}
```

* Response:

```
{
  "response": "The book *The Time Machine* was written by **H. G. Wells** [The Time Machine, Document Body, Page 2]."
}
```

6. DevOps & Deployment Configurations

6.1 Port Mappings & Network Interface

* Port 5173 (React SPA): Exposed on host to serve the browser dashboard UI.

* Port 8000 (FastAPI Server): Exposed on host to allow API calls.

* Network Binding: Vite dev server runs with the `--host` flag to bind to interface `0.0.0.0`, enabling traffic forwarding from your Windows host.

6.2 Host Volume Mounts Layout

Configured inside `docker-compose.yml` to preserve files on disk:

* `./books:/app/books` - Ingest folder.

* `./uploads:/app/uploads` - Staging uploads folder.

* `./database:/app/database` - Stores SQLite metadata records.

* `./chroma_db:/app/chroma_db` - Stores HNSW vector structures.

* `./hf_cache:/app/hf_cache` - Caches Hugging Face model weights.

6.3 Local CPU & Performance Tuning

* Preloaded Weights: Local model parameters are warmed up in RAM at boot time, eliminating the 30-second model loading delay on first query.

* Model Normalization: Preview or deprecated model configurations in `.env` are automatically normalized to `gemini-3.5-flash` or `gemini-3.1-flash-lite` to prevent 404 client crashes.

* Timeout Safety: Stream responses are protected by a 30-second `asyncio.wait_for` wrapper. Spikes in Gemini API traffic trigger clean, user-friendly warnings instead of locking or crashing container resources.